01 LC 104 | Maximum Depth of Binary Tree (Easy)

Problem: Given the root of a binary tree, return its maximum depth. A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.
Constraints: The number of nodes in the tree is in the range [0, 10^4].; -100 <= Node.val <= 100

```
class Solution {
public:
  int maxDepth(TreeNode* root) {
    if (!root) return 0;
    int l = maxDepth(root->left), r =
      maxDepth(root->right);
    return 1 + max(l, r);
  }
};
```

14 LC 236 l Lowest Common Ancestor of a Binarv Tree (Medium)

Problem: Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree. According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”
Constraints: The number of nodes in the tree is in the range [2, 10^5].; -10^9 <= Node.val <= 10^9; All Node.val are unique.; p != q; p and q will exist in the tree.

```
class Solution {
public:
  TreeNode* lowestCommonAncestor(TreeNode* root,
    TreeNode* p, TreeNode* q) {
    if (root == nullptr || root == p || root == q)
      {
      return root;
    }
    auto left = lowestCommonAncestor(root->left,
      p, q);
    auto right = lowestCommonAncestor(root->right,
      p, q);
    if (left && right) {
      return root;
    }
    return left ? left : right;
  }
};
```

23 LC 95 | Unique Binary Search Trees II (Medium)

Problem: Given an integer n, return all the structurally unique BST's (binary search trees), which has exactly n nodes of unique values from 1 to n. Return the answer in any order.
Constraints: 1 <= n <= 8

```
class Solution {
public:
  vector<TreeNode*> generateTrees(int n) {
    function<vector<TreeNode*>(int, int)> dfs =
      [&](int i, int j) {
      if (i > j) {
        return vector<TreeNode*>{nullptr};
      }
      vector<TreeNode*> ans;
      for (int v = i; v <= j; ++v) {
        auto left = dfs(i, v - 1);
        auto right = dfs(v + 1, j);
        for (auto l : left) {
          for (auto r : right) {
            ans.push_back(new TreeNode(v, l, r));
          }
        }
      }
      return ans;
    };
    return dfs(1, n);
  }
};
```

02 LC 94 | Binary Tree Inorder Traversal (Easy)

Problem: Given the root of a binary tree, return the inorder traversal of its nodes' values.
Constraints: The number of nodes in the tree is in the range [0, 100].; -100 <= Node.val <= 100

```
class Solution {
public:
  vector<int> inorderTraversal(TreeNode* root) {
    vector<int> ans;
    while (root) {
      if (!root->left) {
        ans.push_back(root->val);
        root = root->right;
      } else {
        TreeNode* prev = root->left;
        while (prev->right && prev->right != root)
          {
          prev = prev->right;
        }
        if (!prev->right) {
          prev->right = root;
          root = root->left;
        } else {
          ans.push_back(root->val);
          prev->right = nullptr;
          root = root->right;
        }
      }
    }
    return ans;
  }
};
```

25 LC 144 | Binary Tree Preorder Traversal (Easy)

Problem: Given the root of a binary tree, return the preorder traversal of its nodes' values.
Constraints: The number of nodes in the tree is in the range [0, 100].; -100 <= Node.val <= 100

```
class Solution {
public:
  vector<int> preorderTraversal(TreeNode* root) {
    vector<int> ans;
    while (root) {
      if (!root->left) {
        ans.push_back(root->val);
        root = root->right;
      } else {
        TreeNode* prev = root->left;
        while (prev->right && prev->right != root)
          {
          prev = prev->right;
        }
        if (!prev->right) {
          ans.push_back(root->val);
          prev->right = root;
          root = root->left;
        } else {
          prev->right = nullptr;
          root = root->right;
        }
      }
    }
    return ans;
  }
};
```

03 LC 98 | Validate Binary Search Tree (Medium)

Problem: Given the root of a binary tree, determine if it is a valid binary search tree (BST). A valid BST is defined as follows: - The left subtree of a node contains only nodes with keys strictly less than the node's key. - The right subtree of a node contains only nodes with keys strictly greater than the node's key. - Both the left and right subtrees must also be binary search trees.
Constraints: The number of nodes in the tree is in the range [1, 10^4].; -2^31 <= Node.val <= 2^31 - 1

```
class Solution {
public:
  bool isValidBST(TreeNode* root) {
    TreeNode* prev = nullptr;
    function<bool(TreeNode*)> dfs = [&](TreeNode*
      root) {
      if (!root) {
        return true;
      }
      if (!dfs(root->left)) {
        return false;
      }
      if (prev && prev->val >= root->val) {
        return false;
      }
      prev = root;
      return dfs(root->right);
    };
    return dfs(root);
  }
};
```

37 LC 654 | Maximum Binary Tree (Medium)

Problem: Build a binary tree from distinct nums: its root is the maximum value, its left subtree recursively uses values before that maximum, and its right subtree uses those after it. Return the root.
Constraints: 1 <= nums.length <= 1000; 0 <= nums[i] <= 1000; All integers in nums are unique.

```
class Solution {
public:
  TreeNode*
    constructMaximumBinaryTree(vector<int>& nums)
    {
    stack<TreeNode*> stk;
    for (int v : nums) {
      TreeNode* node = new TreeNode(v);
      TreeNode* last = nullptr;
      while (!stk.empty() && stk.top()->val < v) {
        last = stk.top();
        stk.pop();
      }
      node->left = last;
      if (!stk.empty()) {
        stk.top()->right = node;
      }
      stk.push(node);
    }
    while (stk.size() > 1) {
      stk.pop();
    }
    return stk.top();
  }
};
```

04 LC 102 | Binary Tree Level Order Traversal (Medium)

Problem: Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).
Constraints: The number of nodes in the tree is in the range [0, 2000].; -1000 <= Node.val <= 1000

```
class Solution {
public:
  vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> ans;
    if (!root) return ans;
    queue<TreeNode*> q{{root}};
    while (!q.empty()) {
      vector<int> t;
      for (int n = q.size(); n; --n) {
        auto node = q.front();
        q.pop();
        t.push_back(node->val);
        if (node->left) {
          q.push(node->left);
        }
        if (node->right) {
          q.push(node->right);
        }
      }
      ans.push_back(t);
    }
    return ans;
  }
};
```

24 LC 96 | Unique Binary Search Trees (Medium)

Problem: Given an integer n, return the number of structurally unique BST's (binary search trees) which has exactly n nodes of unique values from 1 to n.
Constraints: 1 <= n <= 19

```
class Solution {
public:
  int numTrees(int n) {
    long long count = 1;
    for (int i = 0; i < n; ++i) count = count * 2
      * (2 * i + 1) / (i + 2);
    return static_cast<int>(count);
  }
};
```

32 LC 437 | Path Sum III (Medium)

Problem: Given the root of a binary tree and an integer targetSum, return the number of paths where the sum of the values along the path equals targetSum. The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).
Constraints: The number of nodes in the tree is in the range [0, 1000].; -10^9 <= Node.val <= 10^9; -1000 <= targetSum <= 1000

```
class Solution {
public:
  int pathSum(TreeNode* root, int targetSum) {
    unordered_map<long long, int> cnt;
    cnt[0] = 1;
    auto dfs = [&](auto&& dfs, TreeNode* node,
      long long s) -> int {
      if (!node) {
        return 0;
      }
      s += node->val;
      int ans = cnt[s - targetSum];
      ++cnt[s];
      ans += dfs(dfs, node->left, s) + dfs(dfs,
        node->right, s);
      --cnt[s];
      return ans;
    };
    return dfs(dfs, root, 0);
  }
};
```

05 LC 226 | Invert Binary Tree (Easy)

Problem: Given the root of a binary tree, invert the tree, and return its root.

Constraints: The number of nodes in the tree is in the range [0, 100].; -100 <= Node.val <= 100

```
class Solution {
public:
  TreeNode* invertTree(TreeNode* root) {
    if (!root) {
      return root;
    }
    TreeNode* l = invertTree(root->left);
    TreeNode* r = invertTree(root->right);
    root->left = r;
    root->right = l;
    return root;
  }
};
```

18 LC 297 | Serialize and Deserialize Binary Tree (Hard)

Problem: Implement Codec.serialize(root) and deserialize(data) so every binary tree, including an empty tree, survives a round trip with identical values and structure. You may choose the encoding format.

Constraints: The number of nodes in the tree is in the range [0, 10^4].; -1000 <= Node.val <= 1000

```
class Codec {
public:
  string serialize(TreeNode* root) {
    if (!root) {
      return "";
    }
    queue<TreeNode*> q{{root}};
    string ans;
    while (!q.empty()) {
      auto node = q.front();
      q.pop();
      if (node) {
        ans += to_string(node->val) + " ";
        q.push(node->left);
        q.push(node->right);
      } else {
        ans += "# ";
      }
    }
    ans.pop_back();
    return ans;
  }
  TreeNode* deserialize(string data) {
    if (data == "") {
      return nullptr;
    }
    stringstream ss(data);
    string t;
    ss >> t;
    TreeNode* root = new TreeNode(stoi(t));
    queue<TreeNode*> q{{root}};
    while (!q.empty()) {
      auto node = q.front();
      q.pop();
      ss >> t;
      if (t != "#") {
        node->left = new TreeNode(stoi(t));
        q.push(node->left);
      }
      ss >> t;
      if (t != "#") {
        node->right = new TreeNode(stoi(t));
        q.push(node->right);
      }
    }
    return root;
  }
};
```

06 LC 100 | Same Tree (Easy)

Problem: Given the roots of two binary trees p and q, write a function to check if they are the same or not. Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Constraints: The number of nodes in both trees is in the range [0, 100].; -10^4 <= Node.val <= 10^4

```
class Solution {
public:
  bool isSameTree(TreeNode* p, TreeNode* q) {
    if (p == q) return true;
    if (!p || !q || p->val != q->val) return
      false;
    return isSameTree(p->left, q->left) &&
      isSameTree(p->right, q->right);
  }
};
```

13 LC 235 | Lowest Common Ancestor of a Binary Search Tree (Easy)

Problem: Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST. According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Constraints: The number of nodes in the tree is in the range [2, 10^5].; -10^9 <= Node.val <= 10^9; All Node.val are unique.; p != q; p and q will exist in the BST.

```
class Solution {
public:
  TreeNode* lowestCommonAncestor(TreeNode* root,
    TreeNode* p, TreeNode* q) {
    while (1) {
      if (root->val < min(p->val, q->val)) {
        root = root->right;
      } else if (root->val > max(p->val, q->val))
        {
        root = root->left;
      } else {
        return root;
      }
    }
  }
};
```

42 LC 1026 | Maximum Difference Between Node and Ancestor (Medium)

Problem: Given the root of a binary tree, find the maximum value v for which there exist different nodes a and b where v = |a.val - b.val| and a is an ancestor of b. A node a is an ancestor of b if either: any child of a is equal to b or any child of a is an ancestor of b.

Constraints: The number of nodes in the tree is in the range [2, 5000].; 0 <= Node.val <= 10^5

```
class Solution {
public:
  int maxAncestorDiff(TreeNode* root) {
    int ans = 0;
    function<void(TreeNode*, int, int)> dfs =
      [&](TreeNode* root, int mi, int mx) {
      if (!root) {
        return;
      }
      ans = max({ans, abs(mi - root->val), abs(mx
        - root->val)});
      mi = min(mi, root->val);
      mx = max(mx, root->val);
      dfs(root->left, mi, mx);
      dfs(root->right, mi, mx);
    };
    dfs(root, root->val, root->val);
    return ans;
  }
};
```

07 LC 101 | Symmetric Tree (Easy)
Problem: Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).
Constraints: The number of nodes in the tree is in the range [1, 1000].; -100 <= Node.val <= 100

```
class Solution {
public:
  bool isSymmetric(TreeNode* root) {
    auto dfs = [&](auto&& dfs, TreeNode* root1,
      TreeNode* root2) -> bool {
      if (root1 == root2) {
        return true;
      }
      if (!root1 || !root2 || root1->val !=
        root2->val) {
        return false;
      }
      return dfs(dfs, root1->left, root2->right)
        && dfs(dfs, root1->right, root2->left);
    };
    return dfs(dfs, root->left, root->right);
  }
};
```

12 LC 124 | Binary Tree Maximum Path Sum (Hard)
Problem: Return the maximum sum along any nonempty path in a binary tree. A path uses adjacent nodes, visits no node twice, and need not include the root.
Constraints: The number of nodes in the tree is in the range [1, 3 * 10^4].; -1000 <= Node.val <= 1000

```
class Solution {
public:
  int maxPathSum(TreeNode* root) {
    int ans = -1001;
    function<int(TreeNode*)> dfs = [&](TreeNode*
      root) {
      if (!root) {
        return 0;
      }
      int left = max(0, dfs(root->left));
      int right = max(0, dfs(root->right));
      ans = max(ans, left + right + root->val);
      return root->val + max(left, right);
    };
    dfs(root);
    return ans;
  }
};
```

43 LC 114 | Flatten Binary Tree to Linked List (Medium)
Problem: Given the root of a binary tree, flatten the tree into a "linked list": - The "linked list" should use the same TreeNode class where the right child pointer points to the next node in the list and the left child pointer is always null. - The "linked list" should be in the same order as a pre-order traversal of the binary tree.
Constraints: The number of nodes in the tree is in the range [0, 2000].; -100 <= Node.val <= 100

```
class Solution {
public:
  void flatten(TreeNode* root) {
    while (root) {
      if (root->left) {
        TreeNode* pre = root->left;
        while (pre->right) {
          pre = pre->right;
        }
        pre->right = root->right;
        root->right = root->left;
        root->left = nullptr;
      }
      root = root->right;
    }
  }
};
```

08 LC 543 | Diameter of Binary Tree (Easy)
Problem: Given the root of a binary tree, return the length of the diameter of the tree. The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root. The length of a path between two nodes is represented by the number of edges between them.
Constraints: The number of nodes in the tree is in the range [1, 10^4].; -100 <= Node.val <= 100

```
class Solution {
public:
  int diameterOfBinaryTree(TreeNode* root) {
    int ans = 0;
    auto dfs = [&](auto&& dfs, TreeNode* root) ->
      int {
      if (!root) {
        return 0;
      }
      int l = dfs(dfs, root->left);
      int r = dfs(dfs, root->right);
      ans = max(ans, l + r);
      return 1 + max(l, r);
    };
    dfs(dfs, root);
    return ans;
  }
};
```

49 LC 508 | Most Frequent Subtree Sum (Medium)
Problem: Given the root of a binary tree, return the most frequent subtree sum. If there is a tie, return all the values with the highest frequency in any order. The subtree sum of a node is defined as the sum of all the node values formed by the subtree rooted at that node (including the node itself).
Constraints: The number of nodes in the tree is in the range [1, 10^4].; -10^5 <= Node.val <= 10^5

```
class Solution {
public:
  vector<int> findFrequentTreeSum(TreeNode* root)
    {
    unordered_map<int, int> cnt;
    unordered_map<TreeNode*, int> sub;
    int mx = 0;
    vector<pair<TreeNode*, int>> stk{{root, 0}};
    while (!stk.empty()) {
      auto [node, state] = stk.back();
      stk.pop_back();
      if (!node) {
        continue;
      }
      if (state == 0) {
        stk.emplace_back(node, 1);
        if (node->right) {
          stk.emplace_back(node->right, 0);
        }
        if (node->left) {
          stk.emplace_back(node->left, 0);
        }
        continue;
      }
      int l = node->left ? sub[node->left] : 0;
      int r = node->right ? sub[node->right] : 0;
      int s = node->val + l + r;
      mx = max(mx, ++cnt[s]);
      sub[node] = s;
    }
    vector<int> ans;
    for (const auto& [k, v] : cnt) {
      if (v == mx) {
        ans.push_back(k);
      }
    }
    return ans;
  }
};
```

09 LC 110 | Balanced Binary Tree (Easy)

Problem: Given a binary tree, determine if it is height-balanced.
Constraints: The number of nodes in the tree is in the range [0, 5000].; -10^4 <= Node.val <= 10^4

```
class Solution {
public:
  bool isBalanced(TreeNode* root) {
    function<int(TreeNode*)> height =
      [&](TreeNode* root) {
      if (!root) {
        return 0;
      }
      int l = height(root->left);
      int r = height(root->right);
      if (l == -1 || r == -1 || abs(l - r) > 1) {
        return -1;
      }
      return 1 + max(l, r);
    };
    return height(root) >= 0;
  }
};
```

10 LC 112 | Path Sum (Easy)

Problem: Given the root of a binary tree and an integer targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum. A leaf is a node with no children.
Constraints: The number of nodes in the tree is in the range [0, 5000].; -1000 <= Node.val <= 1000; -1000 <= targetSum <= 1000

```
class Solution {
public:
  bool hasPathSum(TreeNode* root, int targetSum) {
    function<bool(TreeNode*, int)> dfs =
      [&](TreeNode* root, int s) -> int {
      if (!root) return false;
      s += root->val;
      if (!root->left && !root->right && s ==
        targetSum) return true;
      return dfs(root->left, s) ||
        dfs(root->right, s);
    };
    return dfs(root, 0);
  }
};
```

47 LC 337 | House Robber III (Medium)

Problem: Tree nodes represent houses with money. Return the maximum sum robbed without selecting both endpoints of any parent-child edge.
Constraints: The number of nodes in the tree is in the range [1, 10^4].; 0 <= Node.val <= 10^4

```
class Solution {
public:
  int rob(TreeNode* root) {
    function<pair<int, int>(TreeNode*)> dfs =
      [&](TreeNode* root) -> pair<int, int> {
      if (!root) {
        return make_pair(0, 0);
      }
      auto [la, lb] = dfs(root->left);
      auto [ra, rb] = dfs(root->right);
      return make_pair(root->val + lb + rb,
        max(la, lb) + max(ra, rb));
    };
    auto [a, b] = dfs(root);
    return max(a, b);
  }
};
```

11 LC 113 | Path Sum II (Medium)

Problem: Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references. A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.
Constraints: The number of nodes in the tree is in the range [0, 5000].; -1000 <= Node.val <= 1000; -1000 <= targetSum <= 1000

```
class Solution {
public:
  vector<vector<int>> pathSum(TreeNode* root, int
    targetSum) {
    vector<vector<int>> ans;
    vector<int> t;
    function<void(TreeNode*, int)> dfs =
      [&](TreeNode* root, int s) {
      if (!root) return;
      s -= root->val;
      t.emplace_back(root->val);
      if (!root->left && !root->right && s == 0)
        ans.emplace_back(t);
      dfs(root->left, s);
      dfs(root->right, s);
      t.pop_back();
    };
    dfs(root, targetSum);
    return ans;
  }
};
```

22 LC 450 | Delete Node in a BST (Medium)

Problem: Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the root node reference (possibly updated) of the BST. Basically, the deletion can be divided into two stages: - Search for a node to remove. - If the node is found, delete the node.
Constraints: The number of nodes in the tree is in the range [0, 10^4].; -10^5 <= Node.val <= 10^5; Each node has a unique value.; root is a valid binary search tree.; -10^5 <= key <= 10^5

```
class Solution {
public:
  TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return root;
    if (root->val > key) {
      root->left = deleteNode(root->left, key);
      return root;
    }
    if (root->val < key) {
      root->right = deleteNode(root->right, key);
      return root;
    }
    if (!root->left) return root->right;
    if (!root->right) return root->left;
    TreeNode* node = root->right;
    while (node->left) node = node->left;
    node->left = root->left;
    root = root->right;
    return root;
  }
};
```

15 LC 230 | Kth Smallest Element in a BST (Medium)

Problem: Given the root of a binary search tree, and an integer k, return the kth smallest value (1-indexed) of all the values of the nodes in the tree.

Constraints: The number of nodes in the tree is n.; 1 <= k <= n <= 10^4; 0 <= Node.val <= 10^4

```
class Solution {
public:
  int kthSmallest(TreeNode* root, int k) {
    stack<TreeNode*> stk;
    while (root || !stk.empty()) {
      if (root) {
        stk.push(root);
        root = root->left;
      } else {
        root = stk.top();
        stk.pop();
        if (--k == 0) return root->val;
        root = root->right;
      }
    }
    return 0;
  }
};
```

31 LC 129 | Sum Root to Leaf Numbers (Medium)

Problem: You are given the root of a binary tree containing digits from 0 to 9 only. Each root-to-leaf path in the tree represents a number. - For example, the root-to-leaf path 1 -> 2 -> 3 represents the number 123. Return the total sum of all root-to-leaf numbers. Test cases are generated so that the answer will fit in a 32-bit integer. A leaf node is a node with no children.

Constraints: The number of nodes in the tree is in the range [1, 1000].; 0 <= Node.val <= 9; The depth of the tree will not exceed 10.

```
class Solution {
public:
  int sumNumbers(TreeNode* root) {
    function<int(TreeNode*, int)> dfs =
      [&](TreeNode* root, int s) -> int {
      if (!root) return 0;
      s = s * 10 + root->val;
      if (!root->left && !root->right) return s;
      return dfs(root->left, s) + dfs(root->right,
        s);
    };
    return dfs(root, 0);
  }
};
```

48 LC 404 | Sum of Left Leaves (Easy)

Problem: Given the root of a binary tree, return the sum of all left leaves. A leaf is a node with no children. A left leaf is a leaf that is the left child of another node.

Constraints: The number of nodes in the tree is in the range [1, 1000].; -1000 <= Node.val <= 1000

```
class Solution {
public:
  int sumOfLeftLeaves(TreeNode* root) {
    if (!root) {
      return 0;
    }
    int ans = sumOfLeftLeaves(root->right);
    if (root->left) {
      if (!root->left->left && !root->left->right)
        {
        ans += root->left->val;
      } else {
        ans += sumOfLeftLeaves(root->left);
      }
    }
    return ans;
  }
};
```

16 LC 105 | Construct Binary Tree from Preorder and Inorder Traversal (Medium)

Problem: Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Constraints: 1 <= preorder.length <= 3000; inorder.length == preorder.length; -3000 <= preorder[i], inorder[i] <= 3000; preorder and inorder consist of unique values.; Each value of inorder also appears in preorder.; preorder is guaranteed to be the preorder traversal of the tree.; inorder is guaranteed to be the inorder traversal of the tree.

```
class Solution {
public:
  TreeNode* buildTree(vector<int>& preorder,
    vector<int>& inorder) {
    int n = preorder.size();
    unordered_map<int, int> d;
    for (int i = 0; i < n; ++i) {
      d[inorder[i]] = i;
    }
    function<TreeNode*(int, int, int)> dfs =
      [&](int i, int j, int n) -> TreeNode* {
      if (n <= 0) {
        return nullptr;
      }
      int v = preorder[i];
      int k = d[v];
      TreeNode* l = dfs(i + 1, j, k - j);
      TreeNode* r = dfs(i + 1 + k - j, k + 1, n -
        1 - (k - j));
      return new TreeNode(v, l, r);
    };
    return dfs(0, 0, n);
  }
};
```

17 LC 106 | Construct Binary Tree from Inorder and Postorder Traversal (Medium)

Problem: Given two integer arrays inorder and postorder where inorder is the inorder traversal of a binary tree and postorder is the postorder traversal of the same tree, construct and return the binary tree.

Constraints: 1 <= inorder.length <= 3000; postorder.length == inorder.length; -3000 <= inorder[i], postorder[i] <= 3000; inorder and postorder consist of unique values.; Each value of postorder also appears in inorder.; inorder is guaranteed to be the inorder traversal of the tree.; postorder is guaranteed to be the postorder traversal of the tree.

```
class Solution {
public:
  TreeNode* buildTree(vector<int>& inorder,
    vector<int>& postorder) {
    unordered_map<int, int> d;
    int n = inorder.size();
    for (int i = 0; i < n; ++i) {
      d[inorder[i]] = i;
    }
    function<TreeNode*(int, int, int)> dfs =
      [&](int i, int j, int n) -> TreeNode* {
      if (n <= 0) {
        return nullptr;
      }
      int v = postorder[j + n - 1];
      int k = d[v];
      auto l = dfs(i, j, k - i);
      auto r = dfs(k + 1, j + k - i, n - k + i -
        1);
      return new TreeNode(v, l, r);
    };
    return dfs(0, 0, n);
  }
};
```

19 LC 108 | Convert Sorted Array to Binary Search Tree (Easy)
Problem: Given an integer array nums where the elements are sorted in ascending order, convert it to a height-balanced binary search tree.
Constraints: 1 <= nums.length <= 10^4; -10^4 <= nums[i] <= 10^4; nums is sorted in a strictly increasing order.

```
class Solution {
public:
  TreeNode* sortedArrayToBST(vector<int>& nums) {
    auto dfs = [&](auto&& dfs, int l, int r) ->
      TreeNode* {
      if (l > r) {
        return nullptr;
      }
      int mid = (l + r) >> 1;
      return new TreeNode(nums[mid], dfs(dfs, l,
        mid - 1), dfs(dfs, mid + 1, r));
    };
    return dfs(dfs, 0, nums.size() - 1);
  }
};
```

34 LC 617 | Merge Two Binary Trees (Easy)
Problem: Merge two binary trees by summing values where both nodes exist and retaining the existing node where only one exists. Apply this rule recursively and return the merged root.
Constraints: The number of nodes in both trees is in the range [0, 2000].; -10^4 <= Node.val <= 10^4

```
class Solution {
public:
  TreeNode* mergeTrees(TreeNode* root1, TreeNode*
    root2) {
    if (!root1) return root2;
    if (!root2) return root1;
    TreeNode* node = new TreeNode(root1->val +
      root2->val);
    node->left = mergeTrees(root1->left,
      root2->left);
    node->right = mergeTrees(root1->right,
      root2->right);
    return node;
  }
};
```

38 LC 669 | Trim a Binary Search Tree (Easy)
Problem: Trim a BST to retain only nodes with values in inclusive [low,high]. Preserve the relative ancestor/descendant structure of retained nodes; the root may change. Return the trimmed root.
Constraints: The number of nodes in the tree is in the range [1, 10^4].; 0 <= Node.val <= 10^4; The value of each node in the tree is unique.; root is guaranteed to be a valid binary search tree.; 0 <= low <= high <= 10^4

```
class Solution {
public:
  TreeNode* trimBST(TreeNode* root, int low, int
    high) {
    if (!root) return root;
    if (root->val > high) return
      trimBST(root->left, low, high);
    if (root->val < low) return
      trimBST(root->right, low, high);
    root->left = trimBST(root->left, low, high);
    root->right = trimBST(root->right, low, high);
    return root;
  }
};
```

20 LC 700 | Search in a Binary Search Tree (Easy)
Problem: You are given the root of a binary search tree (BST) and an integer val. Find the node in the BST that the node's value equals val and return the subtree rooted with that node. If such a node does not exist, return null.
Constraints: The number of nodes in the tree is in the range [1, 5000].; 1 <= Node.val <= 10^7; root is a binary search tree.; 1 <= val <= 10^7

```
class Solution {
public:
  TreeNode* searchBST(TreeNode* root, int val) {
    if (!root || root->val == val) {
      return root;
    }
    return root->val > val ? searchBST(root->left,
      val) : searchBST(root->right, val);
  }
};
```

21 LC 701 | Insert into a Binary Search Tree (Medium)
Problem: You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST. Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.
Constraints: The number of nodes in the tree will be in the range [0, 10^4].; -10^8 <= Node.val <= 10^8; All the values Node.val are unique.; -10^8 <= val <= 10^8; It's guaranteed that val does not exist in the original BST.

```
class Solution {
public:
  TreeNode* insertIntoBST(TreeNode* root, int val)
    {
    if (!root) {
      return new TreeNode(val);
    }
    if (root->val > val) {
      root->left = insertIntoBST(root->left, val);
    } else {
      root->right = insertIntoBST(root->right,
        val);
    }
    return root;
  }
};
```

44 LC 116 | Populating Next Right Pointers in Each Node (Medium)
Problem: You are given a perfect binary tree where all leaves are on the same level, and every parent has two children. The binary tree has the following definition: struct Node { int val; Node *left; Node *right; Node *next; } Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL. Initially, all next pointers are set to NULL.
Constraints: The number of nodes in the tree is in the range [0, 2^12 - 1].; -1000 <= Node.val <= 1000

```
class Solution {
public:
  Node* connect(Node* root) {
    if (!root) return root;
    root->next = nullptr;
    for (Node* level = root; level->left; level =
      level->left) {
      for (Node* p = level; p; p = p->next) {
        p->left->next = p->right;
        p->right->next = p->next ? p->next->left :
          nullptr;
      }
    }
    return root;
  }
};
```

26 LC 145 | Binary Tree Postorder Traversal (Easy)
Problem: Given the root of a binary tree, return the postorder traversal of its nodes' values.
Constraints: The number of the nodes in the tree is in the range [0, 100].; -100 <= Node.val <= 100

```
class Solution {
public:
  vector<int> postorderTraversal(TreeNode* root) {
    vector<int> ans;
    while (root) {
      if (!root->right) {
        ans.push_back(root->val);
        root = root->left;
      } else {
        TreeNode* next = root->right;
        while (next->left && next->left != root) {
          next = next->left;
        }
        if (next->left != root) {
          ans.push_back(root->val);
          next->left = root;
          root = root->right;
        } else {
          next->left = nullptr;
          root = root->left;
        }
      }
    }
    ranges::reverse(ans);
    return ans;
  }
};
```

46 LC 222 | Count Complete Tree Nodes (Easy)
Problem: Given the root of a complete binary tree, return the number of the nodes in the tree. According to Wikipedia, every level, except possibly the last, is completely filled in a complete binary tree, and all nodes in the last level are as far left as possible. It can have between 1 and 2^h nodes inclusive at the last level h. Design an algorithm that runs in less than O(n) time complexity.
Constraints: The number of nodes in the tree is in the range [0, 5 * 10^4].; 0 <= Node.val <= 5 * 10^4; The tree is guaranteed to be complete.

```
class Solution {
public:
  int countNodes(TreeNode* root) {
    if (!root) {
      return 0;
    }
    int left = depth(root->left);
    int right = depth(root->right);
    if (left == right) {
      return (1 << left) +
        countNodes(root->right);
    }
    return (1 << right) + countNodes(root->left);
  }
  int depth(TreeNode* root) {
    int d = 0;
    for (; root; root = root->left) {
      ++d;
    }
    return d;
  }
};
```

27 LC 199 | Binary Tree Right Side View (Medium)
Problem: Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.
Constraints: The number of nodes in the tree is in the range [0, 100].; -100 <= Node.val <= 100

```
class Solution {
public:
  vector<int> rightSideView(TreeNode* root) {
    vector<int> ans;
    if (!root) {
      return ans;
    }
    queue<TreeNode*> q{{root}};
    while (q.size()) {
      ans.push_back(q.front()->val);
      for (int k = q.size(); k; --k) {
        auto node = q.front();
        q.pop();
        if (node->right) {
          q.push(node->right);
        }
        if (node->left) {
          q.push(node->left);
        }
      }
    }
    return ans;
  }
};
```

35 LC 637 | Average of Levels in Binary Tree (Easy)
Problem: Given the root of a binary tree, return the average value of the nodes on each level in the form of an array. Answers within 10^-5 of the actual answer will be accepted.
Constraints: The number of nodes in the tree is in the range [1, 10^4].; -2^31 <= Node.val <= 2^31 - 1

```
class Solution {
public:
  vector<double> averageOfLevels(TreeNode* root) {
    queue<TreeNode*> q{{root}};
    vector<double> ans;
    while (!q.empty()) {
      int n = q.size();
      long long s = 0;
      for (int i = 0; i < n; ++i) {
        root = q.front();
        q.pop();
        s += root->val;
        if (root->left) {
          q.push(root->left);
        }
        if (root->right) {
          q.push(root->right);
        }
      }
      ans.push_back(s * 1.0 / n);
    }
    return ans;
  }
};
```

28 LC 103 | Binary Tree Zigzag Level Order Traversal (Medium)
Problem: Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).
Constraints: The number of nodes in the tree is in the range [0, 2000].; -100 <= Node.val <= 100

```
class Solution {
public:
  vector<vector<int>> zigzagLevelOrder(TreeNode*
    root) {
    vector<vector<int>> ans;
    if (!root) {
      return ans;
    }
    queue<TreeNode*> q{{root}};
    int left = 1;
    while (!q.empty()) {
      vector<int> t;
      for (int n = q.size(); n; --n) {
        auto node = q.front();
        q.pop();
        t.emplace_back(node->val);
        if (node->left) {
          q.push(node->left);
        }
        if (node->right) {
          q.push(node->right);
        }
      }
      if (!left) {
        ranges::reverse(t);
      }
      ans.emplace_back(t);
      left ^= 1;
    }
    return ans;
  }
};
```

36 LC 662 | Maximum Width of Binary Tree (Medium)
Problem: Return the maximum width across binary-tree levels. Width counts positions from the leftmost to rightmost actual node, including internal gaps in the corresponding complete-tree layout. The answer fits signed 32-bit int.
Constraints: The number of nodes in the tree is in the range [1, 3000].; -100 <= Node.val <= 100

```
class Solution {
public:
  int widthOfBinaryTree(TreeNode* root) {
    if (!root) return 0;
    queue<pair<TreeNode*, uint64_t>> queue;
    queue.emplace(root, 0);
    uint64_t best = 0;
    while (!queue.empty()) {
      uint64_t base = queue.front().second, last =
        0;
      for (int size = queue.size(); size; --size)
        {
        auto [node, position] = queue.front();
          queue.pop();
        position -= base; last = position;
        if (node->left) queue.emplace(node->left,
          2 * position);
        if (node->right)
          queue.emplace(node->right, 2 * position
          + 1);
      }
      best = max(best, last + 1);
    }
    return static_cast<int>(best);
  }
};
```

29 LC 111 | Minimum Depth of Binary Tree (Easy)
Problem: Given a binary tree, find its minimum depth. The minimum depth is the number of nodes along the shortest path from the root node down to the nearest leaf node. Note: A leaf is a node with no children.
Constraints: The number of nodes in the tree is in the range [0, 10^5].; -1000 <= Node.val <= 1000

```
class Solution {
public:
  int minDepth(TreeNode* root) {
    if (!root) {
      return 0;
    }
    queue<TreeNode*> q{{root}};
    int ans = 0;
    while (1) {
      ++ans;
      for (int n = q.size(); n; --n) {
        auto node = q.front();
        q.pop();
        if (!node->left && !node->right) {
          return ans;
        }
        if (node->left) {
          q.push(node->left);
        }
        if (node->right) {
          q.push(node->right);
        }
      }
    }
  }
};
```

50 LC 863 | All Nodes Distance K in Binary Tree (Medium)
Problem: Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node. You can return the answer in any order.
Constraints: The number of nodes in the tree is in the range [1, 500].; 0 <= Node.val <= 500; All the values Node.val are unique.; target is the value of one of the nodes in the tree.; 0 <= k <= 1000

```
class Solution {
public:
  vector<int> distanceK(TreeNode* root, TreeNode*
    target, int k) {
    unordered_map<TreeNode*, TreeNode*> g;
    vector<int> ans;
    auto dfs = [&](auto&& dfs, TreeNode* node,
      TreeNode* fa) {
      if (!node) return;
      g[node] = fa;
      dfs(dfs, node->left, node);
      dfs(dfs, node->right, node);
    };
    auto dfs2 = [&](auto&& dfs2, TreeNode* node,
      TreeNode* fa, int k) {
      if (!node) return;
      if (k == 0) {
        ans.push_back(node->val);
        return;
      }
      for (auto&& nxt : {node->left, node->right,
        g[node]}) {
        if (nxt != fa) {
          dfs2(dfs2, nxt, node, k - 1);
        }
      }
    };
    dfs(dfs, root, nullptr);
    dfs2(dfs2, target, nullptr, k);
    return ans;
  }
};
```

30 LC 257 | Binary Tree Paths (Easy)
Problem: You are given the root of a binary tree. Return all root-to-leaf paths in any order. A leaf is a node with no children.
Constraints: The number of nodes in the tree is in the range [1, 100].; -100 <= Node.val <= 100

```
class Solution {
public:
  vector<string> binaryTreePaths(TreeNode* root) {
    vector<string> ans;
    vector<string> t;
    function<void(TreeNode*)> dfs = [&](TreeNode*
      root) {
      if (!root) {
        return;
      }
      t.push_back(to_string(root->val));
      if (!root->left && !root->right) {
        ans.push_back(join(t));
      } else {
        dfs(root->left);
        dfs(root->right);
      }
      t.pop_back();
    };
    dfs(root);
    return ans;
  }
  string join(vector<string>& t, string sep =
    "->") {
    string ans;
    for (int i = 0; i < t.size(); ++i) {
      if (i > 0) {
        ans += sep;
      }
      ans += t[i];
    }
    return ans;
  }
};
```

41 LC 987 | Vertical Order Traversal of a Binary Tree (Hard)
Problem: Assign root coordinates (row=0,col=0), left child (row+1,col-1), right child (row+1,col+1). Return columns from left to right, with nodes sorted by row, then value for identical positions.
Constraints: The number of nodes in the tree is in the range [1, 1000].; 0 <= Node.val <= 1000

```
class Solution {
public:
  vector<vector<int>> verticalTraversal(TreeNode*
    root) {
    vector<tuple<int, int, int>> nodes;
    function<void(TreeNode*, int, int)> dfs =
      [&](TreeNode* root, int i, int j) {
      if (!root) {
        return;
      }
      nodes.emplace_back(j, i, root->val);
      dfs(root->left, i + 1, j - 1);
      dfs(root->right, i + 1, j + 1);
    };
    dfs(root, 0, 0);
    ranges::sort(nodes);
    vector<vector<int>> ans;
    int prev = -2000;
    for (auto [j, _, val] : nodes) {
      if (j != prev) {
        prev = j;
        ans.emplace_back();
      }
      ans.back().push_back(val);
    }
    return ans;
  }
};
```

33 LC 572 | Subtree of Another Tree (Easy)
Problem: Given the roots of two binary trees root and subRoot, return true if there is a subtree of root with the same structure and node values of subRoot and false otherwise. A subtree of a binary tree tree is a tree that consists of a node in tree and all of this node's descendants. The tree tree could also be considered as a subtree of itself.
Constraints: The number of nodes in the root tree is in the range [1, 2000].; The number of nodes in the subRoot tree is in the range [1, 1000].; -10^4 <= root.val <= 10^4; -10^4 <= subRoot.val <= 10^4

```
class Solution {
  vector<long long> serialize(TreeNode* root) {
    vector<long long> tokens;
    vector<TreeNode*> stack{root};
    while (!stack.empty()) {
      auto node = stack.back(); stack.pop_back();
      if (!node) { tokens.push_back(LLONG_MIN);
        continue; }
      tokens.push_back(node->val);
      stack.push_back(node->right);
        stack.push_back(node->left);
    }
    return tokens;
  }
public:
  bool isSubtree(TreeNode* root, TreeNode*
    subRoot) {
    auto text = serialize(root), pattern =
      serialize(subRoot);
    vector<int> prefix(pattern.size());
    for (int i = 1, j = 0; i <
      static_cast<int>(pattern.size()); ++i) {
      while (j && pattern[i] != pattern[j]) j =
        prefix[j - 1];
      if (pattern[i] == pattern[j]) ++j;
      prefix[i] = j;
    }
    int j = 0;
    for (long long token : text) {
      while (j && token != pattern[j]) j =
        prefix[j - 1];
      if (token == pattern[j]) ++j;
      if (j == static_cast<int>(pattern.size()))
        return true;
    }
    return false;
  }
};
```

39 LC 783 | Minimum Distance Between BST Nodes (Easy)
Problem: Given the root of a Binary Search Tree (BST), return the minimum difference between the values of any two different nodes in the tree.
Constraints: The number of nodes in the tree is in the range [2, 100].; 0 <= Node.val <= 10^5

```
class Solution {
public:
  int minDiffInBST(TreeNode* root) {
    const int inf = 1 << 30;
    int ans = inf, pre = -inf;
    auto dfs = [&](auto&& dfs, TreeNode* root) ->
      void {
      if (!root) {
        return;
      }
      dfs(dfs, root->left);
      ans = min(ans, root->val - pre);
      pre = root->val;
      dfs(dfs, root->right);
    };
    dfs(dfs, root);
    return ans;
  }
};
```

40 LC 938 | Range Sum of BST (Easy)

Problem: Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high].

Constraints: The number of nodes in the tree is in the range [1, 2 * 10^4].; 1 <= Node.val <= 10^5; 1 <= low <= high <= 10^5; All Node.val are unique.

```
class Solution {
public:
  int rangeSumBST(TreeNode* root, int low, int
    high) {
    function<int(TreeNode*)> dfs = [&](TreeNode*
      root) {
      if (!root) {
        return 0;
      }
      int x = root->val;
      int ans = low <= x && x <= high ? x : 0;
      if (x > low) {
        ans += dfs(root->left);
      }
      if (x < high) {
        ans += dfs(root->right);
      }
      return ans;
    };
    return dfs(root);
  }
};
```

45 LC 173 | Binary Search Tree Iterator (Medium)

Problem: Implement BSTIterator(root). next() returns the next smallest BST value; hasNext() reports whether one remains. The initial position is before every value. next/hasNext should be amortized O(1), using O(h) memory; next calls are valid.

Constraints: The number of nodes in the tree is in the range [1, 10^5].; 0 <= Node.val <= 10^6; At most 10^5 calls will be made to hasNext, and next.

```
class BSTIterator {
public:
  std::stack<TreeNode*> nodes;
  BSTIterator(TreeNode* root) {
    for (; root != nullptr; root = root->left) {
      nodes.push(root);
    }
  }
  int next() {
    TreeNode* cur = nodes.top();
    nodes.pop();
    TreeNode* node = cur->right;
    for (; node != nullptr; node = node->left) {
      nodes.push(node);
    }
    return cur->val;
  }
  bool hasNext() {
    return !nodes.empty();
  }
};
```